

# **Progressive Pull Request — SMP integration**

**What, why, file and code, from the corrections to the SMP  
integration**

## Progressive Pull Request — SMP integration into xpack-development

- 1. The pull request at a glance
- 6). Part 0 (#4) is out of band on the port and architecture repos only — it
- 2. Phase 0 — Dissolve the devices repository (PR #4)
  - 3. Phase A — Corrections: single-core correctness (PR #1, Steps 1–13)
  - 4. Phase B — SMP infrastructure (PR #2, Steps 14–23, collapsed into step/14)
  - 5. Phase C1 — Port releases (PR #3, Step 24)
  - 6. Phase C2 — New cores/boards + modular CMake (PR #5, Steps 25–26)
  - 7. Phase C3 — New test sources + add-only platforms (PR #6, Steps 27–28)
  - 8. Phase D — Documentation and final merge (PR #7, Steps 29–30)
  - 9. Per-step commands (the progressive loop)
  - 10. Opening the pull request (manual, by hand)
- 7 --base step/28. #4 (part0-devices) bases on the port branch from #3; it is
- 11. Review and bisect guide
  - 12. Quick reference — what / why / file / code

# Progressive Pull Request — SMP integration

## into `xpack-development`

*What to change, why, in which file, and with which code — step by step, from the corrections to the SMP integration.*

This document turns the internal integration runbook ( `Implementation-SMP-Integration.md` ) and the per-file inventory ( `files-modif-by-step.md` ) into a **progressive pull request**: one reviewable change after another, starting with the single-core corrections and ending with the multi-core (SMP) integration.

Everything below refers to the same model:

- `smp` — the branch that already contains the finished work (source of truth, never modified).
- `xpack-development` — the destination branch the work is rebuilt onto, one cohesive step at a time.
- Every step is **one commit** (tagged `step-NN-green` ) and leaves the tree **green**: the frozen 72-test suite still passes, and for the SMP part the single-core projection is byte-identical to the previous step.

**Table key.** Repo : `K` = `micro-os-plus-iii` (kernel) · `C` = `micro-os-plus-iii-cortexm` · `P` = `micro-os-plus-iii-posix-arch` . **How:** `whole` = `git checkout origin/smp -- <file>` ; `projection` = `git show origin/smp:<file> | scripts/smp/sc-project.py > <file>` (strips the `OS_USE_SMP_SCHEDULER` / `NCPU>1` blocks); `surgical` = insert only the named hunk with `scripts/smp/chunks/_edit.py` . The executable form of every row is `scripts/smp/chunks/stepNN.sh` .

**Synchronised with the branches.** This plan was checked against the actual `smp` and `xpack-development` trees and the recipes in `scripts/smp/chunks/` : every file listed appears in the `xpack-development..smp` diff, and the lift method matches the recipe (`whole` / `sc-project.py` / `_edit.py` ). Three runbook realities are reflected: Part B is **collapsed into a single `step/14` commit** (steps 15–23 never get their own branch); `tests/smp-support/` , `native-smp` and the `test-smp-all` action are **created by the integration**, not present on `smp` ; and the `src/libc/stdlib/timegm.c` change is deliberately **not** lifted (the baseline version is kept).

# 1. The pull request at a glance

| Field           | Value                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| Title           | SMP multi-core integration (progressive, bisectable series)                                                     |
| Repositories    | <code>micro-os-plus-iii</code> · <code>-cortexm</code> · <code>-posix-arch</code> (and the add-only arch repos) |
| Base branch     | <code>xpack-development</code>                                                                                  |
| Head branch     | <code>step/30</code> (or the stacked <code>step/NN</code> chain)                                                |
| Source branch   | <code>smp</code> (read-only)                                                                                    |
| Scope           | Part 0 → Part A (corrections) → Part B (SMP) → Part C (targets) → Part D (docs)                                 |
| Commits         | one per step (Part B is one collapsed <code>step/14</code> commit), each tagged <code>step-NN-green</code>      |
| Acceptance gate | pristine check + 72/72 tests + <code>unifdef</code> invariant (Part B) + <code>test-smp-all</code> (Part C)     |
| Push policy     | nothing is pushed until the PR is opened by hand                                                                |

## 1.1 How the progressive PR is ordered

The order is **corrections** → **SMP evolution** → **SMP tests** → **new targets/tests** → **final integration**, with the two infrastructure items (the `devices` dissolution and the port releases) placed where their dependencies actually are, not first.

| PR | Contents                                                                                                                                               | Steps          | Repos          | Depends on |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|----------------|------------|
| #1 | Corrections (single-core correctness)                                                                                                                  | Part A (1–13)  | K, +C+P at 13  | —          |
| #2 | SMP evolution (collapsed <code>step/14</code> ) + the genuine <code>native-smp</code> dual-core platform, its configs and <code>test-native-smp</code> | Part B (14–23) | K, C, P        | #1         |
| #3 | Port releases: <code>posix-arch v1.1.0</code> , <code>cortexm v1.2.0</code>                                                                            | 24             | P, C           | #2         |
| #4 | Dissolve the <code>devices</code> repository                                                                                                           | Part 0         | P, C, A32, A64 | #3         |
| #5 | New cores/boards + modular CMake                                                                                                                       | 25–26          | K, C, P, arch  | #2, #4     |
| #6 | New test sources + add-only platforms (+ <code>test-smp-all</code> )                                                                                   | 27–28          | K, C, P, arch  | #5         |
| #7 | Documentation + final merge                                                                                                                            | 29–30          | all            | #1–#6      |

PR head branches: `step/13` → `step/14` → `step/24` → `step/26` → `step/28` → `step/30` (the intermediate `step/25` and `step/27` are commits inside PRs #5 and

## 6). Part 0 (#4) is out of band on the port and architecture repos only — it

never touches the kernel — so it is applied on its own `part0-devices` branch before #5 and is not in the kernel's linear chain. Part B is collapsed, so `step/15` ... `step/23` do not exist.

Two equivalent ways to use this:

- **One progressive PR** (recommended for a single reviewer): open #1, and add the later parts as new commits on the same head as they are accepted.
- **Stacked PRs** (recommended for a large review): open #1, then #2 against `step/13`, and so on; each merges into the one below it.

### 1.2 Order verification — is this optimal?

The phase order is forced by the dependency graph. Each claim below was checked against the code, not by analogy.

| Question                                                  | Evidence                                                                                                                                                                                                                                                        | Verdict                                                                   |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Must the corrections come first?                          | Part B's Steps 9/10/13 build on Part A's <code>destroying</code> state, condvar <code>clock</code> and highres-clock port sync; Part A is valuable on its own and shrinks the SMP diff.                                                                         | <b>Yes — first.</b>                                                       |
| Does Part B need Part 0 (dissolve <code>devices</code> )? | Neither port's <code>xpack-development</code> <code>CMakeLists.txt</code> references <code>devices</code> , and <code>part-b.sh</code> fix C deliberately does <b>not</b> apply <code>smp</code> 's <code>UOS_DEVICES_DIR</code> CMake.                         | <b>No.</b> Part 0 only unblocks the new boards, so it moves after Part B. |
| Does Part B have its own test?                            | <code>verify-step.sh</code> Stage 4 is only the NCPU=2 smoke, which the runbook documents as <b>fake</b> (a CMake cache variable never becomes a compile definition); the real <code>native-smp</code> / <code>2xcortex-m33</code> tests only arrive in Part C. | <b>No — the real gap.</b> Ship <code>native-smp</code> with Part B.       |
| Do the new cores/boards need Part B?                      | The M33/RP2350 cores implement the SMP port contract introduced in Part B.                                                                                                                                                                                      | <b>Yes — after Part B.</b>                                                |
| Do the new boards need Part 0?                            | <code>soc/rp2350</code> , <code>soc/stm32f4xx</code> , <code>soc/bcm2837</code> and the shared drivers come from <code>devices</code> .                                                                                                                         | <b>Yes — Part 0 before the new boards.</b>                                |
| Do the tests need the new cores/boards?                   | <code>2xcortex-m33</code> needs the M33 port (Step 25) and the modular CMake (Step 26).                                                                                                                                                                         | <b>Yes — tests after evolutions.</b>                                      |
| Is the final merge last?                                  | Step 30 reconciles with <code>xpack-development</code> and restores <code>.github/**</code> .                                                                                                                                                                   | <b>Yes — last.</b>                                                        |

The original packaging had three non-optimal points, now corrected: Part 0 was first (it is independent of Part A/B), the port releases were buried inside the tests PR (they publish the SMP evolution), and **Part B shipped without a genuine dual-core test** (its only SMP check was the documented-fake smoke). The new `native-smp` leg fixes the last one by landing the real dual-core test with the code it tests.

### 1.3 Is each step self-contained?

Short answer: **each PR is self-contained; each individual step is a self-contained *delta*, not a self-contained *feature*.** A step is green given all the steps before it, but it is not meant to be applied to the baseline on its own. Two forces break standalone independence — **atomic cross-repo/definition steps** and **sequential in-file dependencies** — and Part B is not separable at all.

| Unit                   | Self-contained?                      | Why (evidence)                                                                                                                                                                         |
|------------------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Each <b>PR</b> (#1–#7) | <b>Yes</b> , given its base          | <code>verify-step.sh</code> / <code>full-verify.sh</code> gate it; it builds and passes on the branch it targets.                                                                      |
| Part A Steps 1–12      | Delta only                           | Applied in order; a later step may use an earlier symbol (e.g. Step 9's <code>destroying</code> state).                                                                                |
| Step 13                | Delta + <b>atomic across K, C, P</b> | K declares <code>has_hardware_counter()</code> , C/P define it; landing only the kernel breaks the link (the runbook's <code>qemu-cortex-m7f-release</code> failure).                  |
| Part B (14–23)         | <b>Not separable</b>                 | One interwoven <code>#if defined(OS_USE_SMP_SCHEDULER)</code> blob; <code>part-b.sh</code> states "not per-step bisectable by design"; collapsed into one <code>step/14</code> commit. |
| Step 24                | Yes, given Part B                    | Version-only change ( <code>package.json</code> <code>version</code> ).                                                                                                                |
| Step 25                | Yes, given Part B + Part 0           | New port files; atomic across C/P.                                                                                                                                                     |
| Step 26                | <b>Needs 25</b>                      | The M33 platform links the M33 port target.                                                                                                                                            |
| Step 27                | <b>Needs 26</b>                      | The new suites are wired into the new M33 platform.                                                                                                                                    |
| Step 28                | <b>Needs 26–27</b>                   | The AArch platforms use the harness and tests added before.                                                                                                                            |
| Part 0                 | Yes (port/arch only)                 | Independent of the kernel; its gate is "boards that linked <code>devices</code> still build".                                                                                          |

Why some steps cannot stand alone:

- **Atomic definition + declaration.** Step 13 adds `port::clock_highres:: has_hardware_counter()` in the kernel and its definition in the two ports; the two halves must be committed together or the link fails. Part B (Step 14) and Step 25 are the same: the kernel's `port_cpu_id()` and the port's definition, or the M33 core and its CMake target, travel together.
- **In-file chaining.** `os-c-wrapper.cpp` is touched by Steps 4, 9 and 20; `os-thread.{h,cpp}` by 1, 7, 9, 14 and 20; `os-c-decls.h` by 9, 10 and 16; `os-core.cpp` by 14–22. The recipes use **whole** / **projection** / **surgical** lifting so each step carries only *its* hunks — but it still assumes the earlier steps are already in place (see §11.2).
- **Deliberate coupling.** Step 4 strips the Step-9 `destroying` `static_assert` because it does not compile yet; Step 7 keeps Step-9/20 content out to avoid `-Werror=switch-enum`. The steps are curated to *avoid* pulling the future in, not to make each one buildable from the baseline.

The rule the recipes enforce: a step may lift a **partial file**, but never a **partial dependency** — a declaration and its definition, or a test and the platform it runs on, always travel in the same step (or PR).

---

## 2. Phase 0 — Dissolve the `devices` repository (PR #4)

---

**Why the whole part.** `xpack-development` has six components; `smp` adds a seventh, `micro-os-plus-iii-devices`. Upstream has no such repo and nowhere to put one. Folding its code (history-preserving `git subtree`) into the architecture repos that already own it lets the rest of the integration proceed with the six repos upstream has.

| Repo           | What                                  | Why                                                                          | File / path                                                                             | How (code)                                                                              |
|----------------|---------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| P              | Move the native SoC                   | host SD-image backend belongs to the host port                               | <code>soc/native/</code>                                                                | <code>git subtree add -P soc/native &lt;devices&gt; split/soc_native</code>             |
| P              | Move the shared drivers               | block/FatFs/USB drivers are portable C++, used by boards                     | <code>drivers/include/</code> , <code>drivers/src/</code> , <code>drivers/fatfs/</code> | <code>git subtree add -P drivers/... &lt;devices&gt; split/...</code>                   |
| P              | Carry driver fixes                    | atomic-spinlock mailbox; flatfs append-to-empty allocates an extent          | <code>src/.../mailbox</code> , <code>fatfs</code>                                       | part of the subtree move                                                                |
| C              | Move the STM32F4xx SoC                | Cortex-M silicon owned by the Cortex-M port                                  | <code>soc/stm32f4xx/</code>                                                             | <code>git subtree add -P soc/stm32f4xx &lt;devices&gt; split/soc_stm32f4xx</code>       |
| C              | Move the RP2350 SoC                   | Pico 2 headers + <code>system_rp2350.c</code>                                | <code>soc/rp2350/</code>                                                                | <code>git subtree add -P soc/rp2350 &lt;devices&gt; split/soc_rp2350</code>             |
| A32/A64        | Move drivers + <code>bcm2837</code>   | each arch repo must be self-contained (no cross-repo dep)                    | <code>drivers/</code> , <code>soc/bcm2837/</code>                                       | <code>git subtree add ...</code>                                                        |
| P, C, A32, A64 | Re-declare the old CMake target names | so every board that linked <code>micro-os-plus::devices</code> keeps working | <code>CMakeLists.txt</code> (shim)                                                      | <code>add_library(... INTERFACE) + add_library(micro-os-plus::devices ALIAS ...)</code> |

**Gate for PR #4:** every board that linked a `devices` target still configures and builds; the standalone `devices` repo is no longer a dependency.

---

### 3. Phase A — Corrections: single-core correctness (PR #1, Steps 1–13)

---

**Why the whole part.** These fixes are correct and valuable **without** SMP. Landing them first keeps each step bisectable and makes Part B a pure SMP diff.

### 3.1 Steps

| Step | What                                                                    | Why                                                                                                                                                                                                                     | File                                                                                                                                                                                                                                                                                                                    | Code / how                                                                                                                                                                                                                                                                                                                                                                                 | Gate                                               |
|------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| 1    | ISO C conformance, list iterators, exported suspend, newlib return type | host glibc double-declares <code>DIR</code> ; intrusive-list iterators must conform; the C wrapper must resolve <code>thread::suspend</code> ; newlib's <code>read / write</code> return type must match the weak alias | <code>include/cmsis-plus/posix/dirent.h</code> ,<br><code>include/cmsis-plus/utils/lists.h</code> ,<br><code>include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h</code> ,<br><code>src/rtos/os-thread.cpp</code>                                                                                                  | whole for the three headers — <code>c-syscalls-aliases-standard.h</code> defines <code>CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE</code> as newlib's <code>_READ_WRITE_RETURN_TYPE</code> (or <code>ssize_t</code> ); surgical: remove <code>inline</code> from <code>this_thread::suspend</code> in <code>os-thread.cpp</code> .<br><b>Keep baseline</b><br><code>src/libc/stdlib/timegm.c</code> | 72/72                                              |
| 2    | Memory: overflow, usable size, calloc/ realloc, block-pool              | header decl + base definition + override must land together or <code>memory_resource::do_usable_size</code> is undefined; overflow guards                                                                               | <code>include/cmsis-plus/rtos/os-memory.h</code> ,<br><code>include/cmsis-plus/memory/first-fit-top.h</code> , <code>src/rtos/os-memory.cpp</code> ,<br><code>src/memory/first-fit-top.cpp</code> , <code>src/memory/lifo.cpp</code> , <code>src/memory/block-pool.cpp</code> , <code>src/libc/stdlib/malloc.cpp</code> | whole (7 files); add <code>-Wcast-align / -Wunsafe-buffer-usage</code> pragma wrap in <code>first-fit-top.cpp</code> ; calloc guard <code>if (nelem != 0 &amp;&amp; elbytes &gt; (SIZE_MAX / nelem)) { errno = ENOMEM; return nullptr; }</code>                                                                                                                                            | 72/72; <code>mutex-stress</code> + allocator paths |
| 3    | C++17 aligned new/delete, static error category, chrono                 | aligned allocation; exit-time-destructor warning; 64-bit overflow                                                                                                                                                       | <code>src/libcpp/new.cpp</code> , <code>src/libcpp/system-error.cpp</code> ,<br><code>src/libcpp/chrono.cpp</code>                                                                                                                                                                                                      | whole; add clang <code>-Wexit-time-destructors</code> suppression for the two Meyers-singleton categories;<br><code>(cyc/f)*1e9 + (cyc%f)*1e9/f</code>                                                                                                                                                                                                                                     | 72/72                                              |
| 4    | C wrapper: one-shot timer, polymorphic                                  | correct timer default; delete the right type; cast to                                                                                                                                                                   | <code>src/rtos/os-c-wrapper.cpp</code>                                                                                                                                                                                                                                                                                  | whole, then <b>strip</b> Step-9 <code>destroying</code>                                                                                                                                                                                                                                                                                                                                    | 72/72                                              |

| Step | What                                                                           | Why                                                                                   | File                                                                                                                                                                                                                         | Code / how                                                                                                                                                                                              | Gate  |
|------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
|      | delete, 64-bit timeouts                                                        | 64-bit before <code>*</code><br><code>1000u</code>                                    |                                                                                                                                                                                                                              | static_assert and Step-20 SMP affinity blocks                                                                                                                                                           |       |
| 5    | POSIX I/O thread-safety                                                        | descriptor-table race; free-list race; inverted size check                            | <code>src/posix-io/file-descriptors-manager.cpp</code> ,<br><code>include/cmsis-plus/posix-io/file-system.h</code> ,<br><code>include/cmsis-plus/posix-io/net-stack.h</code> ,<br><code>src/posix-io/block-device.cpp</code> | whole; add <code>-Wunsafe-buffer-usage</code> suppression in the fd manager; <code>if (size == 0) return EINVAL;</code>                                                                                 | 72/72 |
| 6    | ARMv8-M guards, SecureFault, semihosting fstat                                 | M33 build; fault handler;<br><code>S_IFCHR</code> only when no type set               | <code>include/cmsis-plus/arm/semihosting.h</code> ,<br><code>src/startup/exception-handlers.c</code> ,<br><code>src/semihosting/c-syscalls-semihosting.cpp</code>                                                            | whole; add <code>__ARM_ARCH_8M_MAIN__ / __ARM_ARCH_8M_BASE__</code> guards + <code>SecureFault_Handler</code> ; add weak <code>os_board_console_mirror()</code>                                         | 72/72 |
| 7    | Timer callback outside the critical section                                    | avoid running user code while the lock is held; periodic catch-up re-arm              | <code>src/rtos/internal/os-lists.cpp</code> ,<br><code>src/rtos/os-timer.cpp</code> ,<br><code>include/cmsis-plus/rtos/os-thread.h</code>                                                                                    | whole for the two sources; <b>surgical</b> errno-scratch hunk into <code>os-thread.h</code> (keep Step-9/20 out)                                                                                        | 72/72 |
| 8    | Mutex priority ceiling                                                         | set ceiling before ownership; boost to max waiter; recompute owner priority on unlock | <code>src/rtos/os-mutex.cpp</code>                                                                                                                                                                                           | whole                                                                                                                                                                                                   | 72/72 |
| 9    | Thread lifecycle: <code>state::destroying</code> , atomic join, detach, reaper | race-free destruction/join; single-lock register/suspend                              | <code>include/cmsis-plus/rtos/os-thread.h</code> , <code>src/rtos/os-thread.cpp</code> ,<br><code>src/rtos/os-idle.cpp</code> ,<br><code>include/cmsis-plus/rtos/os-c-decls.h</code>                                         | <b>projection</b> for the three ( <code>os-thread.cpp</code> has ~37 SMP hunks); <b>surgical</b> <code>destroying = 7</code> into <code>os-c-decls.h</code> (keep Step-10 <code>void* clock</code> out) | 72/72 |

| Step | What                                                               | Why                                                                                                            | File                                                                                                                                                                                                  | Code / how                                                                                                                                                                            | Gate  |
|------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 10   | Condition-variable atomicity                                       | link-then-unlock-then-suspend; bind <code>timed_wait</code> to the attribute clock                             | <code>src/rtos/os-condvar.cpp</code> , <code>include/cmsis-plus/rtos/os-condvar.h</code> , <code>include/cmsis-plus/diag/instrumentation.h</code> , <code>include/cmsis-plus/rtos/os-c-decls.h</code> | whole for the three; <b>surgical</b> : uncomment only <code>os_condvar_t</code> 's <code>void* clock</code> ; add <code>OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR (12)</code>  | 72/72 |
| 11   | <code>std::thread</code> functor lifetime & join                   | join the native handle before delete; deterministic free                                                       | <code>src/libcpp/thread-cpp.h</code> , <code>include/cmsis-plus/estd/thread_internal.h</code>                                                                                                         | whole                                                                                                                                                                                 | 72/72 |
| 12   | Message-queue reschedule                                           | preempt after a send/receive wakes a higher-priority thread                                                    | <code>src/rtos/os-mqueue.cpp</code>                                                                                                                                                                   | whole; <code>port::scheduler::reschedule()</code> after the critical section in all 6 paths                                                                                           | 72/72 |
| 13   | High-resolution clock port sync (first cross-repo step: K + C + P) | declare/call <code>has_hardware_counter()</code> ; fix SysTick pending test; read <code>CLOCK_MONOTONIC</code> | K: <code>include/cmsis-plus/rtos/os-decls.h</code> , <code>src/rtos/os-clocks.cpp</code> ; C: <code>include/cmsis-plus/rtos/port/os-inlines.h</code> ; P: same file                                   | K whole; C <b>projection</b> ( <code>has_hardware_counter()→false</code> , <code>hardware_counter()→0</code> ); P <b>surgical</b> insert of only the <code>clock_highres</code> block | 72/72 |

**Step 1 completeness note.** The runbook's detailed Step-1 table also lists `include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h`; the current `chunks/step01.sh` lifts only `dirent.h`, `lists.h` and the surgical `os-thread.cpp` edit. Include the aliases header when building this PR to cover the newlib `_READ_WRITE_RETURN_TYPE` fix (it is a real `xpack-development..smp` change).

### 3.2 The one code fix that is easy to get wrong (Step 13)

```
// C: include/cmsis-plus/rtos/port/os-inlines.h
inline clock_highres::timestamp_t clock_highres::cycles_since_tick (void) {
    uint32_t load = SysTick->LOAD, val = SysTick->VAL;
    if ((SCB->ICSR & SCB_ICSR_PENDSTSET_Msk) && (val > (load / 2)))
        return (load - val) + load;
    return load - val;
}
```

---

## 4. Phase B — SMP infrastructure (PR #2, Steps 14–23, collapsed into [step/14](#))

---

**Why one blob.** Unlike Part A, all ten steps' SMP code is interleaved inside the same `#if defined(OS_USE_SMP_SCHEDULER)` blocks in the same files. The whole set lands as **one commit** ([step/14](#), recipe [chunks/part-b.sh](#)) across the three repos, then the hardening fixes A–J are applied. **Every** change is inside the `OS_USE_SMP_SCHEDULER` guard, so the single-core binary is unchanged (proved by `unifdef`). The ten rows below are the conceptual sub-steps of that one commit; steps 15–23 do not exist as separate branches.

## 4.1 The SMP steps

| Step | What                                                              | Why                                                                            | File                                                                                                                                    | Code / how                                                                                                                                                                 |
|------|-------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 14   | <code>port_cpu_id()</code>                                        | identify the current core; TLS on the host, constant 0 on single-core Cortex-M | K: <code>src/rtos/os-core.cpp</code> , <code>os-thread.cpp</code> ; C: <code>os-inlines.h</code> ; P: <code>src/rtos/os-core.cpp</code> | P: <code>thread_local unsigned _this_cpu = 0; extern "C" __attribute__((noinline)) unsigned port_cpu_id(void) { asm volatile(":::"memory"); return _this_cpu; }</code>     |
| 15   | Per-CPU scheduler lock<br><code>lock_state[OS_NCPU]</code>        | one scheduler lock per core                                                    | K+P+C<br><code>src/rtos/os-core.cpp</code>                                                                                              | scalar → array; in P mask <code>SIGALRM</code> before reading <code>port_cpu_id()</code>                                                                                   |
| 16   | Recursive kernel lock<br><code>_smp_klock</code>                  | serialize kernel entry/exit across cores                                       | <code>include/cmsis-plus/rtos/port/os-decls.h</code> , <code>os-c-decls.h</code>                                                        | <code>struct smp_klock_t { atomic lock; owner; count; }; _smp_klock_enter/exit</code> ; clear owner <b>before</b> the release store; <code>SMP_NO_OWNER 0xFFFFFFFFU</code> |
| 17   | Per-CPU interrupt tracking                                        | know if the current core is in handler mode                                    | P <code>os-decls.h</code>                                                                                                               | <code>irq_set + _in_isr[OS_NCPU]</code> ; <code>in_handler_mode()</code> reads <code>_in_isr[port_cpu_id()]</code>                                                         |
| 18   | Per-CPU current thread<br><code>current_thread[OS_NCPU]</code>    | each core has its own running thread                                           | <code>os-sched.h</code>                                                                                                                 | array; <code>this_thread::thread()</code> reads <code>[port_cpu_id()]</code> with local IRQs masked                                                                        |
| 19   | Per-CPU idle threads<br><code>os_idle_thread_core[OS_NCPU]</code> | every core needs an idle context                                               | K+C+P<br><code>os-idle.cpp</code>                                                                                                       | each core registers its private idle context at boot                                                                                                                       |
| 20   | CPU affinity mask                                                 | pin threads to cores; main thread to core 0                                    | <code>os-thread.cpp</code>                                                                                                              | <code>th_cpu_affinity</code> (default <code>0xFFFFFFFF</code> ), <code>cpu_affinity()</code> API, main pinned to <code>1U&lt;&lt;0</code>                                  |
| 21   | 5-stage deferred publish/claim                                    | hand a thread between cores without losing the stack pointer                   | K+C+P                                                                                                                                   | claim ( <code>to-&gt;stack_ptr=nullptr</code> ) → spill → SP switch → publish ( <code>from-&gt;stack_ptr=sp</code> ,                                                       |

| Step | What                                | Why                                                              | File                                                                          | Code / how                                                                                                           |
|------|-------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
|      |                                     |                                                                  |                                                                               | release) → restore;<br>picker gate<br>state_ != running<br>&& stack_ptr !=<br>nullptr                                |
| 22   | SMP ready-list picker               | choose the highest-<br>priority runnable thread<br>for this core | os-core.cpp<br>internal_switch_th<br>reads                                    | (affinity &<br>(1<cpu)) and<br>published stack; else<br>os_idle_thread_cor<br>e[cpu]                                 |
| 23   | POSIX host<br>multiprocessing / IPI | model cores as<br>pthreads; wake a core<br>by signal             | P src/<br>host_cpu.cpp ,<br>include/<br>host_cpu.hpp ; K<br>weak port_smp_ipi | spawn OS_NCPU<br>pthreads; per-CPU<br>timer_create(SIGEV<br>_THREAD_ID) ; IPI via<br>pthread_kill(..,<br>SIGRTMIN+1) |

## 4.2 The files lifted for Part B

| Repo | Files (whole, SMP versions)                                                                                                                                                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| K    | include/cmsis-plus/rtos/os-c-decls.h ,<br>os-sched.h , os-thread.h ; src/rtos/os-c-<br>wrapper.cpp , os-core.cpp , os-idle.cpp , os-<br>main.cpp , os-thread.cpp                                                                                               |
| C    | include/cmsis-plus/rtos/port/os-c-decls.h , os-<br>decls.h , os-inlines.h ; src/rtos/os-core.cpp                                                                                                                                                               |
| P    | include/cmsis-plus/rtos/port/os-c-decls.h , os-<br>decls.h , os-inlines.h ; include/host_cpu.hpp ,<br>exception_handler.hpp , hw_result.hpp ; src/<br>host_cpu.cpp , free-store.cpp ,<br>board-contract.cpp , exception_handler.cpp , src/<br>rtos/os-core.cpp |

### 4.3 The lift-time corrections (fixes A–J)

| Fix | What                                                                                                                            | Why                                                                                                     | File                                                       | Code                                                                                                                                                                                                               |
|-----|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A/B | <code>struct timespec</code><br><code>tp;</code> → <code>timespec</code><br><code>tp;</code>                                    | baseline gate is<br><code>-Werror=redundant-</code><br><code>tags</code>                                | P <code>os-inlines.h</code>                                | drop the <code>struct</code> tag                                                                                                                                                                                   |
| C   | Register new SMP<br>sources <b>additively</b>                                                                                   | keep the xpack<br>INTERFACE model,<br>not smp's standalone<br><code>UOS_SMP_DIR</code>                  | P <code>CMakeLists.txt</code>                              | append to<br><code>target_sources(...</code><br><code>INTERFACE)</code> ; do <b>not</b><br>build <code>board-</code><br><code>contract.cpp</code> here                                                             |
| D   | Register<br><code>host_cpu.cpp</code> ,<br><code>free-store.cpp</code> ,<br><code>exception_handler.</code><br><code>cpp</code> | the refactored always-<br>compiled <code>os-</code><br><code>core.cpp</code> delegates<br>to them       | P <code>CMakeLists.txt</code>                              | <code>target_sources(...</code><br><code>INTERFACE src/</code><br><code>host_cpu.cpp src/</code><br><code>free-store.cpp</code><br><code>src/</code><br><code>exception_handler.</code><br><code>cpp)</code>       |
| E   | Drop redundant<br><code>port_cpu_id</code> decl;<br>declare<br><code>port_smp_ipi</code>                                        | <code>-Werror=missing-</code><br><code>declarations</code>                                              | P <code>host_cpu.cpp</code>                                | remove dup decl; add<br>prior prototype for the<br>strong override                                                                                                                                                 |
| F/G | Remove redundant<br><code>port_cpu_id</code> decl;<br>weak<br><code>g_core_stage[]</code>                                       | <code>-Werror=redundant-</code><br><code>decls</code> ; symbol only<br>defined by the SMP<br>test board | P<br><code>exception_handler.</code><br><code>cpp</code>   | add a port-side weak<br>definition of<br><code>g_core_stage[OS_NC</code><br><code>PU]</code>                                                                                                                       |
| H   | Wrap kernel<br><code>port_cpu_id</code><br>redundant decls                                                                      | <code>-Werror=redundant-</code><br><code>decls</code> only under a<br>true SMP compile                  | K <code>os-core.cpp</code> ,<br><code>os-thread.cpp</code> | <code>#pragma GCC</code><br><code>diagnostic ignored</code><br><code>"-Wredundant-</code><br><code>decls"</code>                                                                                                   |
| I/J | clang<br><code>-Weverything</code><br>hardening                                                                                 | destination is stricter<br>than <code>smp</code>                                                        | P port headers + SMP<br><code>.cpp</code>                  | add <code>-Wc++98-</code><br><code>compat-pedantic</code> /<br><code>-Wreserved-</code><br><code>identifier</code> /<br><code>-Wunsafe-buffer-</code><br><code>usage</code> ; suppression<br>block before includes |

### 4.4 The dual-core test that must ship with Part B (`native-smp`)

Part B's own gate (`verify-step.sh` Stage 4) is only a smoke build, and the runbook records that the original NCPU=2 command-line build was **fake** — the `-D` values became CMake cache variables, not compile definitions, so the code built single-core and the "SMP" run proved nothing. The only way SMP is truly on is a platform that sets the defines via `target_compile_definitions`. The `native-smp` platform does exactly that, so it belongs in PR #2:

| What                                     | Why                                                                    | File                                                                                                                                                  | Code                                                                                                                                 |
|------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Add the <code>native-smp</code> platform | a genuine dual-core host platform is the only real SMP test for Part B | <code>tests/platforms/native-smp/</code> (mirrors <code>native</code> ; links <code>micro-os-plus::iii-posix-arch</code> + <code>libucontext</code> ) | <code>target_compile_definitions(... PRIVATE OS_INTEGER_RTOS_PORT_NCPU=2 OS_USE_SMP_SCHEDULER=1)</code>                              |
| Add its configs and actions              | run it from xpm                                                        | <code>tests/package.json</code> (append-only)                                                                                                         | <code>native-smp-cmake-gcc14-debug/release</code> ; <code>test-native-smp</code> ; introduce <code>test-smp-all</code> with this leg |
| Wire it into the harness                 | select the platform                                                    | <code>tests/cmake/tests-main.cmake</code> (additive)                                                                                                  | platform chosen by <code>PLATFORM_NAME</code>                                                                                        |

This is the change that surfaced kernel fix **H** (the `port_cpu_id` redundant-decl clash under a real SMP compile), already in `part-b.sh`. `2xcortex-m33` stays in PR #6 because it also needs the M33 port (Step 25) and the modular CMake (Step 26).

**Gate for PR #2:** pristine check + `unifdef -UOS_USE_SMP_SCHEDULER` single-core delta = 0 + 72/72 + the genuine `native-smp` dual-core suite ( `rtos-apis-test`, `mutex-stress-test`, `cmsis-os-validator-test` ).

## 5. Phase C1 — Port releases (PR #3, Step 24)

**Why now.** The releases publish the SMP port model that Part B just landed; they depend only on Part B, not on the new boards or tests, so they are their own small PR rather than being buried in the tests PR.

| Step | What          | Why                                                                                         | File / path                                                | How                                                                                                                                                                         |
|------|---------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 24   | Port releases | publish the SMP port model ( <code>posix-arch v1.1.0</code> , <code>cortexm v1.2.0</code> ) | P/C <code>package.json</code> ( <code>version</code> only) | <code>release-port.sh</code> edits <code>version</code> in Python and tags by hand — <b>never</b> <code>npm version</code> , which runs <code>postversion</code> and pushes |

**Gate for PR #3:** the release commits touch only the `version` line; both tags exist locally only; the port branches still build.

## 6. Phase C2 — New cores/boards + modular CMake (PR #5, Steps 25–26)

**Why here.** These are **evolutions**: the M33 and RP2350 cores and the additive CMake that lets a new platform link the fat `::iii` without touching the existing targets. They need Part B (the SMP port contract) and Part 0 (the SoC code), and they unblock the tests in PR #6.

| Step | What                                 | Why                                                                  | File / path                                                                                                                                                                                                                                                                                         | How                                                                                                                                                |
|------|--------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 25   | New cores/boards                     | M33 and RP2350 support                                               | C <code>include-m33/</code> ,<br><code>include-rp2350/</code> ,<br><code>os-core-m33.cpp</code> ,<br><code>os-core-rp2350.cpp</code> ; P<br><code>board-contract.cpp</code> ,<br><code>free-store.cpp</code> ,<br><code>exception_handler.*</code> ; K IPI in<br><code>thread::resume()</code>      | add-only from<br><code>origin/smp</code>                                                                                                           |
| 26   | Modular CMake (integrated, add-only) | build the new M33 target without breaking the fat <code>::iii</code> | C <code>CMakeLists.txt</code> (additive <code>micro-os-plus::cortexm-qemu-m33</code> ); K<br><code>CMakeLists.txt</code> (additive harness sub-targets); <code>tests/platforms/2xcortex-m33/</code> ; <code>tests/device-qemu-cortexm-m33/</code> ; <code>tests/package.json</code> config + action | append the port target; new platform links the fat <code>::iii</code> ; self-contained device;<br><code>mps2-an521 --cpu cortex-m33 --smp 2</code> |

**Gate for PR #5:** the original `xpm run test-all` (72) still green; the new M33 platform configures and builds.

## 7. Phase C3 — New test sources + add-only platforms (PR #6, Steps 27–28)

**Why here.** These are the **tests**: existing, proven `smp` suites promoted add-only, plus the AArch platforms. They need the cores/boards from PR #5.

| Step | What                                  | Why                                | File / path                                                                                                                                                                                                                                                                     | How                                                                                                                                                                                                                                     |
|------|---------------------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 27   | New test sources                      | prove SMP on the new architectures | <code>tests/sources/fp-switch/</code> ; <code>tests/smp-support/</code> (dissolved <code>test_smp1/</code> ); smp tests 0–5; AN505/AN521 linker scripts                                                                                                                         | <code>absorb-test-smpl.sh</code> ; append <code>set(ENABLE_FP_SWITCH_TEST true)</code> to <code>global-definitions.cmake</code>                                                                                                         |
| 28   | Add-only platforms and harness wiring | run the suites on new targets      | <code>tests/platforms/2xcortex-m33</code> , <code>cortexm-pico2</code> , <code>aarch32-rpi3b</code> , <code>aarch64-rpi3b</code> (the <code>native-smp</code> leg already landed in PR #2); <code>tests/cmake/tests-main.cmake</code> (additive); K <code>CMakeLists.txt</code> | <code>integrate-aarch-harness.sh</code> adds <code>iii-posix-io</code> / <code>iii-semihosting</code> / <code>iii-newlib-reent</code> / <code>test-support</code> sub-targets; extend <code>test-smp-all</code> with the M33/aarch legs |

Gate for PR #6: the original `xpm run test-all` (72) still green and `xpm run test-smp-all` green.

## 8. Phase D — Documentation and final merge (PR #7, Steps 29–30)

| Step | What                | Why                                          | File                                                                                | Code                                                                                                                                                                 |
|------|---------------------|----------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 29   | Documentation       | keep the runbook/PDF in step with the code   | <code>docs/*.md</code> , <code>docs/*.pdf</code> , <code>docs/render-pdfs.sh</code> | <code>cd docs &amp;&amp; ./render-pdfs.sh</code><br><code>Implementation-SMP-Integration</code>                                                                      |
| 30   | Final merge (local) | reconcile with upstream and drop dev tooling | all repos; remove <code>scripts/smp/</code>                                         | merge <code>origin/xpack-development</code> ; restore <code>.github/</code> , <code>README</code> , <code>LICENSE</code> ; <code>finalize.sh</code> ; run both gates |

Gate for PR #7: clean doc build; full ecosystem `test-all` + `test-smp-all`; `.github/` `README` / `LICENSE` byte-identical to baseline.

## 9. Per-step commands (the progressive loop)

One step = four commands. Nothing is pushed.

| # | Command                                                                                                                 | What it does                                                                                                        |
|---|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1 | <code>scripts/smp/new-step.sh NN</code>                                                                                 | fork <code>step/NN</code> from the most recent existing step branch (or the repo's <code>xpack-development</code> ) |
| 2 | <code>scripts/smp/chunks/stepNN.sh</code><br>(or <code>chunks/part-b.sh</code> for the collapsed <code>step/14</code> ) | apply the exact files + corrections for the step                                                                    |
| 3 | <code>scripts/smp/verify-step.sh NN</code>                                                                              | pristine check → <code>unifdef</code> (Part B / <code>step/14</code> ) → link ports → 72/72 tests                   |
| 4 | <code>scripts/smp/advance-step.sh NN</code><br><code>"message"</code>                                                   | commit the touched repos and tag <code>step-NN-green</code>                                                         |

Drive Part A and the collapsed Part B with `scripts/smp/run-loop.sh 1 14`, then Part C with `scripts/smp/run-loop.sh 24 30`; finish with `full-verify.sh` (mandatory), then `finalize.sh`.

## 10. Opening the pull request (manual, by hand)

Nothing is automated up to this point. When the series is green locally, open the progressive PR yourself:

```
git -C "$WORK/micro-os-plus-iii" push origin step/30

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head step/30 \
  --title "SMP multi-core integration (progressive, bisectable series)" \
  --body-file docs/pool-request.md
```

For the stacked variant, open each against the previous head: #2 `--base step/13`, #3 `--base step/14`, #5 `--base step/24`, #6 `--base step/26`,

## 7 `--base step/28`. #4 (`part0-devices`) bases on the port branch from #3; it is

out of band, so #5's port side is rebased on it before review.

## 11. Review and bisect guide

| Reviewer question                              | Where to look                                                | What proves it                                                                                          |
|------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Did the test framework stay frozen?            | <code>scripts/smp/check-pristine.sh</code>                   | existing <code>package.json</code> / <code>tests/**/.github/**</code> entries never modified or deleted |
| Is the single-core binary unchanged in Part B? | <code>verify-step.sh</code> Stage 1                          | <code>unifdef -UOS_USE_SMP_SCHEDULER</code> projection is identical to the previous step                |
| Does every step build and pass?                | <code>verify-step.sh NN</code>                               | 72/72 (24 builds × 3 tests) with <code>-Werror</code>                                                   |
| Which commit introduced a regression?          | <code>git bisect</code> over <code>step-NN-green</code> tags | each tag is a labelled green checkpoint                                                                 |
| Is the full ecosystem green?                   | <code>full-verify.sh</code>                                  | real <code>xpm install → link-deps → prepare → build → test</code> per config                           |

### 11.1 Pass criteria per part

| Part   | Steps | Pristine rule                                 | Extra gate                                                                       |
|--------|-------|-----------------------------------------------|----------------------------------------------------------------------------------|
| Part A | 1–13  | total freeze                                  | 72/72                                                                            |
| Part B | 14–23 | total freeze                                  | 72/72 + <code>unifdef</code> invariant + <code>native-smp</code> dual-core suite |
| Part C | 24–28 | append-only                                   | 72/72 + <code>test-smp-all</code>                                                |
| Part D | 29–30 | not gated (restores <code>.github/**</code> ) | doc build + both suites                                                          |

### 11.2 Files that carry several steps (lift carefully)

| File                                              | Steps                  | Watch for                                                                                                             |
|---------------------------------------------------|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>src/rtos/os-c-wrapper.cpp</code>            | 4, 9, 20               | <code>os_thread_state_destroying</code> , SMP affinity <code>#if</code>                                               |
| <code>src/rtos/os-thread.cpp</code>               | 1, 9, 14, 20           | exported symbol (1), destroying/join (9), <code>port_cpu_id</code> /affinity (14, 20)                                 |
| <code>include/cmsis-plus/rtos/os-thread.h</code>  | 1, 7, 9, 14, 20        | errno hunk (7), destroying enum (9), affinity (14, 20)                                                                |
| <code>include/cmsis-plus/rtos/os-c-decls.h</code> | 9, 10, 16              | <code>destroying = 7</code> (9), <code>os_condvar_t</code> <code>void* clock</code> (10), <code>smp_klock</code> (16) |
| <code>src/rtos/os-core.cpp</code>                 | 14, 15, 16, 18, 19, 22 | almost entirely Part B, all under <code>#if</code>                                                                    |

## 12. Quick reference — what / why / file / code

| Phase  | What                                      | Why                                                | Files                                                                                        | Code (representative)                                                                                                   |
|--------|-------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Part 0 | Fold <code>devices</code> into arch repos | upstream has no seventh repo                       | <code>soc/*</code> , <code>drivers/*</code> , CMake shims                                    | <code>git subtree add -P &lt;path&gt; &lt;devices&gt; split/&lt;sub&gt;</code>                                          |
| Part A | 13 single-core fixes                      | correct and valuable without SMP; keep Part B pure | <code>include/**</code> , <code>src/**</code> , ports                                        | <code>git checkout origin/smp -- &lt;files&gt; / sc-project.py / _edit.py</code>                                        |
| Part B | SMP scheduler, locks, IPI                 | multi-core support                                 | <code>os-core.cpp</code> , <code>os-sched.h</code> , port headers, <code>host_cpu.cpp</code> | inside <code>#if defined(OS_USE_SMP_SCHEDULER)</code> ; <code>_smp_klock</code> , <code>current_thread_[OS_NCPU]</code> |
| Part C | New targets and tests                     | add-only coverage for M33/RP2350/AArch             | <code>tests/platforms/**</code> , <code>tests/sources/**</code>                              | append <code>test-smp-all</code> ; <code>target_compile_definitions(... OS_USE_SMP_SCHEDULER=1)</code>                  |
| Part D | Docs + final merge                        | land the series cleanly                            | <code>docs/**</code> , <code>.github/**</code>                                               | <code>finalize.sh</code> ; open PR by hand                                                                              |

**The one rule that keeps the series trustworthy:** never define `OS_USE_SMP_SCHEDULER` on the `cmake / xpm` command line (a cache variable does not become a compile flag). It must come from a platform's `target_compile_definitions`, and you must see it on the real compile line ( `cmake --build <dir> -v` ). Otherwise the "SMP" build is silently single-core.